Real-Time IoT Sensor Monitoring, Cloud Storage and Predictive Analytics Framework

Athanasios Tasis
athanasios@tasis.info
MEng Computer Engineering & Informatics Department
University of Patras

1. Remote Setup

Due to remote collaboration requirements, a shared development environment was designed based on an *Ubuntu 16.04 Virtual Machine* installed on a central computer located in Patras. All team members connected through the same *TailScale mesh VPN network*, allowing secure system access without requiring port forwarding.

This architecture provided:

- secure remote SSH access,
- static virtual IP addressing,
- shared file access through *sshfs*,

This approach enabled complete remote development and project management without requiring physical presence in the same location.

2. Sensor Initialization and Configuration

2.1 TelosB Configuration

After installing the TinyOS environment, the TinyOS sensing applications were compiled and deployed on the TelosB motes. The serial ports were identified using:

```
ls /dev | grep ttyUSB*
```

The original sensing example was then modified to support periodic data acquisition.

The main modifications included:

- sampling interval set to 20 s,
- addition of a battery sensor field,
- extension of the logger from 4 to 5 sensors,
- integration of the `BatteryVoltageC` component.

The TelosB operates as a sensing node and collects measurements of temperature, humidity, light intensity, and battery voltage.

2.2 IRIS Base Station

The IRIS mote was used as a base station and packet forwarder. Using the tool:

```
java net.tinyos.sf.SerialForwarder
```

incoming USB serial packets are converted into TCP/IP packets on port 9002, enabling live streaming of sensor data through a network socket.

The raw ADC values were converted into real measurement units using the equations provided in the hardware manual:

$$hum = -4.0 + 0.0405 \cdot humRaw - 2.8 \times 10^{-6} \cdot humRaw^2$$

$$temp_C = -39.6 + 0.01 \cdot tempRaw$$

$$battery_V = \frac{1.5 \cdot 4096.0}{batteryRaw}$$

3. Data Collection Pipeline

For measurement management, a custom Java application based on `SensingApp.java` was developed. Each measurement packet is initially stored locally in JSON format.

Each record contains:

- timestamp,
- node identifier,
- temperature,
- humidity,
- light intensity,
- solar intensity,
- battery voltage.

The data are then transmitted to a cloud-hosted MongoDB Atlas collection. This cloud-based approach allows all team members to remotely access the stored data up to the 512MB free-tier limit.

4. Visualization and Monitoring

Grafana was used for the visualization layer together with the MongoDB datasource plugin. Multiple live dashboards were developed to display the latest measurements, historical database entries, and derived metrics such as:

$$\frac{light}{temperature}, \qquad \frac{temperature}{humidity}, \qquad \frac{solar}{light}$$

Custom MongoDB aggregation queries were created for each Grafana panel.

Figure 1: Grafana dashboard

4.1 Feature Selection

For feature selection, a threshold range of:

$$0.99 > threshold > 0.88$$

was defined. The top 20 features were then ranked and selected.

5. Intelligence

5.1 Preprocessing

After data collection, preprocessing was applied. Specifically:

- Cross-checking database measurements against the local `.json` file.
- Keeping the first occurrence in duplicate entries.
- Detecting missing measurements using `timestamp.diff > 24`.
- Detecting timestamps with differences smaller than 17 seconds.
- Applying a hardcoded upper limit of 150 lux for light measurements.
- Computing z-scores and defining a threshold of 3 standard deviations from the mean.
- Detecting 534 outliers using IQR with threshold 2.2 instead of 1.5.
- Applying interpolation for NaN values.
- Smoothing the data using a moving average over 9 measurements.

5.2 Feature Extraction

For the creation of derived features, the following were computed:

`Rolling, Lag, IQR, Skew(1hr), Kurtosis(1hr), sin/cos(hour), phase of day, heat_index`

using window sizes of 1, 5, 15, 30, 60, and 180 samples.

Next, the average score from Spearman correlation, Pearson correlation, mutual information, random forest importance, and ANOVA was calculated. A custom scoring metric was then created and used to generate the correlation heatmap.

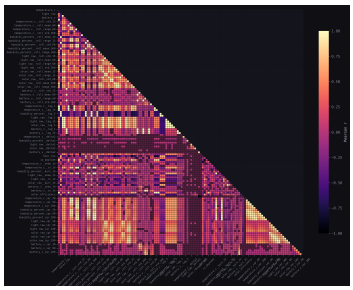

Figure 2: Correlation Heatmap

5.3 Multiple Linear Regression

5.3.1 Train/Test

For improved hyperparameter tuning, the dataset was divided into 6 folds while progressively revealing more data to the model. The size of each test set was calculated as:

$$\text{Test Size} = \frac{16580 \text{ entries}}{6+1} = 2368$$

5.3.2 Evaluation

The metrics used were:

`MAE, MSE, Train` R^2 `- Test` R^2 `(overfitting metric).`

5.4 SARIMAX

The data were sorted chronologically, and the variables `temperature_c`, `humidity_percent`, and `light_raw` were selected as targets. Lag features from previous measurements were used as exogenous variables.

For evaluation, `TimeSeriesSplit` with 6 folds was applied in order to preserve the temporal ordering of the data. For each

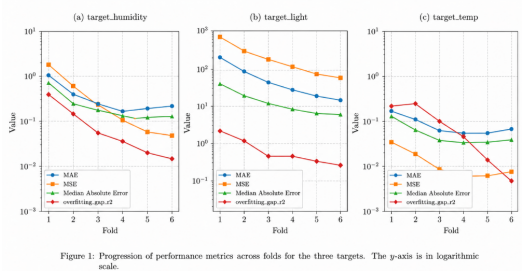

Figure 3: Fold Metrics

target, a `SARIMAX(2,0,1)` model was trained and forecasting was performed on the corresponding test set.

Finally, the metrics `MAE`, `MSE`, `RMSE`, and `R²` were calculated, while the results were exported to an Excel file for further analysis.

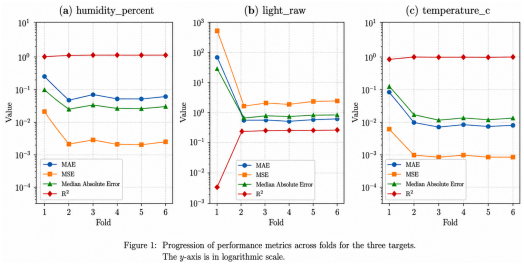

Figure 4: Fold Metrics

5.5 SARIMAX vs. Regression

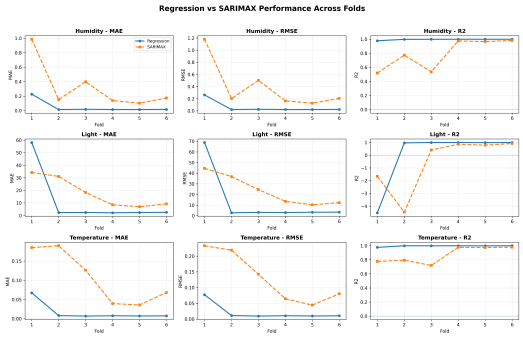

Figure 5: Fold Metrics

6. Conclusion

Abstract

This work presents a complete IoT system based on TelosB and IRIS motes that collects temperature, humidity, light intensity, and battery voltage measurements every 20 s. The data are transmitted through a Java application to MongoDB Atlas and visualized in real time using Grafana. After preprocessing and feature extraction, predictive models were trained with a forecasting horizon of 100 timesteps.

7. Links

- Grafana Dashboard: athanasios.grafana.net
- GitHub Repository: github.com/thonos-cpu/Temp_Hum_Prediction_Model
- Overleaf Source: main.tex